

Autonomous 6WD Rover Architecture for Environmental Hexagonal Mapping and Cellular Automata Wildfire Simulation

The proliferation of autonomous ground vehicles within environmental monitoring frameworks has introduced unprecedented capabilities for real-time, in-situ data acquisition across hazardous and inaccessible topographies. The proposed architecture describes a highly complex cyber-physical system: a six-wheel-drive (6WD), 360-degree vertical rotation rover governed by a Raspberry Pi 4 microprocessor. This platform is specifically designed to navigate unstructured natural environments, aggregate microclimate and topographical data, identify dry, combustible vegetation via edge-computed computer vision, and synthesize this telemetry to construct a top-down hexagonal spatial grid. This geospatial data structure subsequently serves as the foundational matrix for running localized, real-time fire propagation simulations predicated on stochastic Cellular Automata (CA) mathematical models.

This comprehensive report provides an exhaustive evaluation of the hardware feasibility, mechatronic kinematics, sensor integration methodologies, geospatial data structuring, and mathematical simulation modeling required to instantiate this system. By synthesizing advanced inverse kinematics, embedded systems engineering, and geospatial informatics, the subsequent analysis delineates the operational parameters, physical limitations, and computational frameworks necessary for successful deployment.

1. Kinematics and Mechatronics of the 6WD Swerve Drive Architecture

The mechanical foundation of the rover relies on a highly articulated drivetrain designed for extreme mobility, continuous traction, and vector-agnostic translation. The requirement for a "6WD 360-wheel vertical rotation rover" implies a specialized swerve-drive configuration, potentially mounted on a rocker-bogie or similar articulated suspension system to facilitate stair-climbing or obstacle negotiation in wildland environments.¹

1.1 Holonomic Mobility and Inverse Kinematics

Traditional skid-steer systems suffer from high lateral friction, unpredictable odometry drift, and severe terrain tearing when turning in dense vegetation. In contrast, utilizing six independent GA25-370 DC gear motors for propulsion, paired with six MG90S micro servos for 360-degree vertical axis rotation (azimuth control), grants the rover true holonomic mobility. This mechanical paradigm permits the vehicle to translate along any vector without altering the absolute heading of the chassis, a critical capability for maintaining specific camera

orientations toward target vegetation while navigating complex inclines.²

To achieve fluid speed and movement demonstration, the Raspberry Pi 4 must continuously calculate the inverse kinematics for all six independent wheel pods. For a given desired chassis translational velocity vector characterized by longitudinal velocity V_x and lateral velocity V_y , along with a desired rotational velocity around the chassis center of mass ω , the required steering angle δ_i and drive wheel velocity v_i for each wheel pod i (located at geometric coordinates x_i, y_i relative to the center of mass) must be computed in real-time. The kinematic equations governing this transformation are defined as:

$$v_{x,i} = V_x - \omega \cdot y_i$$

$$v_{y,i} = V_y + \omega \cdot x_i$$

From these individual wheel vectors, the commanded drive velocity and servo steering angle are derived:

$$v_i = \sqrt{v_{x,i}^2 + v_{y,i}^2}$$

$$\delta_i = \arctan2(v_{y,i}, v_{x,i})$$

Executing this matrix of equations continuously demands a highly optimized software loop on the Raspberry Pi 4, generating the target setpoints for the motor controllers.

1.2 Actuator Specifications and Terrain Negotiation

The GA25-370 micro gear motors are particularly well-suited for rugged terrain traversal.³ Assuming a standard configuration operating at 6V to 12V with a 1:46.8 gear ratio, these motors provide an optimal balance between rotational velocity (typically around 130 RPM under nominal load) and the torque required to overcome steep gradients.³ The high reduction ratio acts as an inherent mechanical brake, preventing the rover from sliding backward when navigating the steep elevation gradients detected by the onboard inertial measurement unit. The integration of MG90S metal-gear micro servos for wheel pod rotation introduces a challenge regarding stall torque and holding current. While the MG90S provides superior durability compared to plastic-gear alternatives (such as the SG90), the mechanical linkage must be designed to minimize lateral sheer forces on the servo spline.⁵ Thrust bearings must be implemented above each wheel pod to support the weight of the chassis, ensuring the servo only exerts rotational force rather than bearing vertical loads.

2. Electronic Topology and Multiplexing Feasibility Analysis

The control of six DC motors and six servo motors alongside a dense sensor array presents a significant input/output (I/O) bottleneck for a standard Raspberry Pi 4. The microprocessor possesses hardware limitations regarding the generation of stable, jitter-free Pulse Width Modulation (PWM) signals across multiple pins, necessitating external hardware acceleration.⁶

2.1 The PCA9685 Constraint and I2C Daisy-Chaining

The architectural requirement specifies the utilization of a "16-channel servo controller (controlling both servos and L298N inputs)." The industry standard for this application is the PCA9685 16-channel 12-bit PWM/Servo driver communicating via the I2C bus.⁷ However, an exhaustive pin-mapping analysis reveals a critical mathematical constraint that renders a single 16-channel board insufficient for the proposed task.

The hardware manifest dictates the control of the following actuators:

1. **Six MG90S Servos:** Requiring 6 discrete PWM channels.
2. **Three L298N Motor Drivers:** Each L298N controls two DC motors. A single L298N module requires two PWM enable pins (ENA, ENB) for precise speed regulation and four logic pins (IN1, IN2, IN3, IN4) for directional control (forward, reverse, brake).⁹ Controlling three L298N drivers therefore requires 6 PWM pins and 12 directional logic pins, totaling 18 control signals.

Attempting to route all 6 servo signals and all 18 L298N inputs through a single PCA9685 requires 24 channels, which exceeds the IC's 16-channel physical limit.¹⁰ To fulfill the architectural mandate of utilizing the controller for *both* the servos and the L298N inputs, the system must employ a multiplexed daisy-chain topology.

The PCA9685 natively supports I2C address cascading.⁸ By bridging the A0 solder pad on a second PCA9685 board, its I2C address shifts from the default 0x40 to 0x41.¹² This expands the system capacity to 32 independent PWM channels using only the two standard I2C pins (SDA and SCL) on the Raspberry Pi 4.⁸

Implementing two independent PCA9685 controllers also resolves a fundamental frequency conflict. Servos such as the MG90S strictly require a 50Hz to 60Hz PWM frequency (a 20ms frame rate) to interpret the 1ms to 2ms pulse widths correctly.⁵ Conversely, driving DC motors via the L298N at 50Hz results in audible coil whining and inefficient torque transfer; DC motors perform optimally at PWM frequencies between 1kHz and 10kHz.⁶ Because the PCA9685 features a single global prescaler, all 16 channels on a single board must share the same frequency.⁵ Daisy-chaining permits Board A (Address 0x40) to operate at 50Hz for the steering servos, while Board B (Address 0x41) operates at 2kHz for the L298N ENA/ENB and directional inputs, ensuring optimal electromechanical efficiency across all propulsion systems.¹²

2.2 Analog-to-Digital Conversion (ADC) Integration

The hardware specification correctly identifies the necessity of an Analog-to-Digital Converter (ADC). Unlike many microcontrollers, the Broadcom System-on-Chip (SoC) utilized in the Raspberry Pi 4 completely lacks native analog inputs. The inclusion of the MQ135 gas sensor, which outputs a continuous analog voltage proportional to target gas concentrations, absolutely mandates this intermediary hardware.¹⁴

A high-resolution ADC, such as the ADS1115 (16-bit, I2C interface) or the MCP3008 (10-bit, SPI interface), must be integrated into the sensor bus. The ADC will continuously sample the analog voltage output from the MQ135 and any other analog instrumentation (such as a

potentiometer if utilized in early wind vane prototypes), digitizing the waveforms into discrete integer values that the Python backend can subsequently parse into meaningful environmental metrics.

2.3 Power Distribution and Isolation

The integration of these electromechanical components necessitates a rigorous, multi-rail power distribution architecture. The Raspberry Pi 4 requires a stable 5V/3A supply. The six MG90S servos operate optimally at 5V, but under stall conditions during steering maneuvers on rough terrain, six servos can draw cumulative transient currents exceeding 5 Amperes.¹⁷

Drawing this current from the Pi's 5V GPIO rail will instantly induce a brownout, causing catastrophic system resets.¹⁹

Furthermore, the GA25-370 motors driven by the L298N modules require a dedicated higher-voltage supply, such as an 11.1V (3S) Lithium Polymer (LiPo) battery. The L298N utilizes bipolar junction transistors (BJTs) arranged in Darlington pairs rather than modern MOSFETs, resulting in a substantial and unavoidable voltage drop of approximately 1.4V to 2.0V across the internal H-bridge.⁹ Consequently, an 11.1V input will deliver approximately 9.1V to the motors. Opto-isolation and strict ground unification strategies must be meticulously implemented. The PCA9685 intentionally separates its logic power (VCC, 3.3V supplied by the Raspberry Pi) from the high-current servo power (V+, supplied by a dedicated 5V buck converter stepping down the main LiPo battery) to protect the sensitive I2C bus from inductive voltage spikes generated by the motors.⁸ A unified common ground plane across all batteries, motor drivers, and the microprocessor is mandatory to provide a stable reference voltage for logic signaling.¹⁷

Power Subsystem	Component Target	Voltage Requirement	Recommended Source
Logic Bus	Raspberry Pi 4, Sensors, PCA9685 (VCC)	5.0V / 3.3V	5V / 3A Dedicated UBEC
Servo Bus	6x MG90S, PCA9685 (V+)	5.0V	5V / 10A Dedicated Buck Converter
Motor Bus	3x L298N, 6x GA25-370	7.4V - 11.1V	Direct 2S or 3S LiPo Battery

3. Environmental Sensor Array and Vector Logic

The accuracy of the subsequent wildfire propagation simulation is strictly dependent on the fidelity and resolution of the real-time empirical data collected by the rover's diverse sensor suite. Each sensor provides a specific variable required by the mathematical models governing the Cellular Automata grid.

3.1 Custom Anemometer Kinematics and Logic

Wind speed and wind direction represent the most dominant meteorological variables driving

the rate and direction of fire propagation.²¹ The system employs a DIY anemometer and wind vane constructed using four SS41F Hall-effect sensors. The SS41F is a bipolar Hall-effect digital switch; it outputs a low signal (active LOW) when its sensing element is exposed to a magnetic field of sufficient flux density, and returns to a HIGH state when the field is removed.²³

Wind Speed (Anemometer): The cup anemometer utilizes a rotating vertical shaft fitted with a diametrically magnetized neodymium disc. As the hemispherical cups are driven by the aerodynamic drag of the wind, the magnet repeatedly passes a fixed SS41F sensor, triggering a square-wave pulse train.²³ The Raspberry Pi monitors this sensor via a hardware interrupt pin, bypassing the CPU-intensive polling cycle. The wind speed v is calculated using the frequency of the detected pulses f , the radius of the anemometer arms r , and an empirically derived calibration coefficient C that accounts for aerodynamic drag coefficients and bearing friction:

$$v = 2 \pi r \cdot f \cdot C$$

By counting the pulses over a fixed temporal gate (e.g., an integration period of 2.0 seconds), the system derives the localized, near-ground wind velocity in meters per second.²⁷

Wind Direction (Wind Vane): Measuring wind direction across a full 360-degree spectrum using only four discrete digital sensors requires a specialized mechanical and logical implementation. The four SS41F sensors are positioned orthogonally (North, East, South, West) around a central shaft connected to the aerodynamic wind vane.²⁹

By utilizing a magnet with a field wide enough to trigger two adjacent sensors simultaneously, the system achieves an 8-point compass resolution (45-degree angular increments) from only four inputs.³⁰ The Python logic matrix evaluates the active LOW signals from the sensor array to determine the prevailing wind azimuth:

North Sensor	East Sensor	South Sensor	West Sensor	Detected Direction	Azimuth Vector (θ)
0	1	1	1	North	0°
0	0	1	1	North-East	45°
1	0	1	1	East	90°
1	0	0	1	South-East	135°
1	1	0	1	South	180°
1	1	0	0	South-West	225°
1	1	1	0	West	270°
0	1	1	0	North-West	315°

(Note: Logic 0 indicates magnetic flux detection due to the active LOW open-collector architecture of the SS41F).³⁰ This 8-point vector is critical for establishing the propagation heading of the simulated fire front.

3.2 Microclimate and Fuel Moisture Indexing

The MQ135 air quality gas sensor and the DHT11 digital temperature and humidity sensor

provide localized microclimate characterization.¹⁴ The MQ135 relies on a stannic oxide (SnO_2) semiconductor layer whose electrical conductivity increases in the presence of target gases.¹⁶ It is highly sensitive to ammonia, sulfides, benzene, and carbon dioxide (CO_2). The analog output voltage of the MQ135 is processed through the aforementioned ADC, and the resulting digital value is mathematically converted to a relative Parts Per Million (PPM) reading using the standard R_s/R_0 resistance ratio formula. This serves as a secondary validation tool, allowing the rover to detect the invisible chemical precursors of combustion before visual flames are apparent.¹⁴

The DHT11 data is utilized to approximate the Fine Fuel Moisture Code (FFMC) or similar components of the Fire Weather Index (FWI).³² The moisture content of dead grass and fine vegetative fuels responds rapidly to changes in ambient relative humidity and temperature.³⁴ High ambient temperatures coupled with low relative humidity exponentially increase the combustibility of the vegetation detected by the vision system. The relative humidity (RH) and temperature (T) values are directly fed into the mathematical probability modifiers of the simulation's cellular automata rules.³⁵

3.3 Topographical Mapping via IMU and GPS Sensor Fusion

Topography, specifically the slope and aspect of the terrain, profoundly influences thermodynamic fire propagation; fires burn significantly faster uphill due to convective heat transfer pre-heating the adjacent unburned fuel ahead of the advancing flame front.³⁶ To map this crucial parameter, the rover utilizes a Neo 6M GPS module and an MPU6050 6-axis Inertial Measurement Unit (IMU).³⁸

The Neo 6M GPS provides macroscopic global coordinates (Latitude, Longitude), but commercial GPS inherently lacks the vertical precision required for micro-topographical elevation mapping.³⁹ The MPU6050, combining a 3-axis Micro-Electromechanical Systems (MEMS) accelerometer and a 3-axis gyroscope, bridges this gap.³⁸

Because raw accelerometer data is highly susceptible to the high-frequency vibrational noise of the 6WD chassis traversing rough ground, and raw gyroscope data suffers from integration drift over time, the system must employ a Complementary Filter or an Extended Kalman Filter (EKF) algorithm to fuse the sensor telemetry.⁴¹ The standard complementary filter for calculating the pitch angle (θ_{pitch}) operates on the principle of trusting the gyroscope for short-term dynamic changes and the accelerometer for long-term static gravity alignment:

$$\theta_{\text{pitch}} = 0.98 \cdot (\theta_{\text{pitch}} + \omega_y \cdot dt) + 0.02 \cdot \theta_{\text{accel}_y}$$

The fused data yields highly accurate roll (ϕ) and pitch (θ) angles relative to the gravity vector.⁴⁴ By treating the rover as a mobile terrain probe, the system projects this gravity vector against the rover's localized frame of reference.⁴⁵ As the rover traverses the GPS-defined bounding box, the continuous aggregation of these surface normals allows the Raspberry Pi to reconstruct a localized elevation gradient map, successfully identifying ridges,

slopes, and valleys which are subsequently encoded as the elevation parameters (Z_{elev}) in the hexagonal grid.⁴⁶

4. Computer Vision for Biomass and Fuel Characterization

The identification of combustible biomass, specifically desiccated or dry grass, is achieved via the onboard USB camera utilizing the OpenCV (Open Source Computer Vision) library running directly on the Raspberry Pi 4.⁴⁸

4.1 Edge Computing Limitations and Deterministic Algorithms

While deploying convolutional neural networks (e.g., YOLOv10, MobileNet, or TensorFlow Lite models) for advanced object detection is feasible on the Raspberry Pi 4, the intense computational overhead associated with deep learning inference can induce rapid thermal throttling, drastically reducing the video frame rate to unmanageable levels for real-time robotic navigation.⁵⁰ Therefore, optimized, deterministic algorithmic approaches utilizing color space transformations and mathematical morphological operations are highly preferred for continuous, low-latency background scanning.⁵²

4.2 Color Space Segmentation and the NGRDI Vegetation Index

Traditional RGB (Red, Green, Blue) image matrices are highly susceptible to variations in natural illumination, such as shifting shadows, cloud cover, and solar glare.⁵⁴ To reliably detect dry grass regardless of lighting conditions, the raw image matrices are converted into the HSV (Hue, Saturation, Value) color space.⁵² The Hue channel effectively isolates chromatic information independent of the lighting intensity represented by the Value channel. Dry, cured grass exhibits specific spectral signatures predominantly in the yellow-to-brown hue ranges.⁵⁵ By applying strictly calibrated upper and lower HSV boundary thresholds, a binary mask is generated where active pixels represent highly combustible dry biomass.⁵²

To mathematically augment the robustness of the detection without requiring expensive Near-Infrared (NIR) multi-spectral cameras, the system applies the Normalized Green Red Difference Index (NGRDI) directly to the visible-spectrum pixels.⁴⁹ The NGRDI is computed dynamically across the image array using the raw digital numbers from the camera's sensor:

$$\text{NGRDI} = \frac{\text{Green} - \text{Red}}{\text{Green} + \text{Red}}$$

Healthy, photosynthetic vegetation strongly reflects green wavelengths and absorbs red wavelengths for chlorophyll synthesis, yielding positive NGRDI values. Conversely, senescent, dry, or dead vegetation—which represents an extreme fire risk—yields negative values due to the total breakdown of chlorophyll and the emerging spectral dominance of red-reflecting carotenes and xanthophylls.⁴⁹

The resulting segmented data from the OpenCV pipeline is quantified as a continuous "Fuel Density and Combustibility Factor." This percentage-based metric is temporally tagged with the

current GPS coordinates and the IMU-derived elevation normal, creating a discrete spatial data packet ready for mapping.

5. Telemetry and Semantic Data Transmission via LoRa

The inclusion of the RF96 LoRa (Long Range) module introduces highly reliable, extended-distance telemetry capabilities to the rover.⁵⁸ LoRa transceivers typically operate on sub-gigahertz radio frequency bands (e.g., 433 MHz, 868 MHz, or 915 MHz depending on local geographic regulations) and utilize Chirp Spread Spectrum (CSS) modulation. This physical layer protocol provides exceptional signal resilience against electromagnetic interference, multi-path fading, and physical obstruction by dense foliage.⁵⁹

5.1 Bandwidth Limitations and Semantic Encoding

While LoRa technology can achieve line-of-sight communication distances exceeding 15 to 20 kilometers in ideal environments, it is fundamentally engineered as a low-power, low-data-rate protocol.⁵⁹ A critical physical limitation must be explicitly acknowledged in the system architecture: streaming raw video data from the USB camera over a LoRa link is mathematically impossible.⁶¹ LoRa data rates typically maximize at a few kilobits per second (kbps) under optimal spreading factors, whereas even highly compressed, low-resolution video streams require bandwidths in the megabits per second (Mbps) range.

Therefore, the rover architecture must utilize the concept of "semantic data transmission." Instead of transmitting raw pixel arrays, the Raspberry Pi 4 leverages its edge-computing capabilities to process the camera and sensor data locally.⁶² The output of this processing—specifically, the binary flags indicating the presence of dry grass, the computed NGRDI indices, the 8-point wind vector, ambient temperature, humidity, and precise GPS coordinates—are serialized into lightweight telemetry packets (typically formatted as compact JSON strings or Protocol Buffers). This highly condensed semantic payload, usually ranging from 50 to 250 bytes, can be easily and continuously transmitted over the RF96 module to a remote base station for real-time visualization and simulation oversight.³⁰

6. Hexagonal Geospatial Data Structuring

The continuous aggregation of the telemetry, topographical, and visual data forms the environmental map. The system requirement specifies the generation of a "top-down hexagonal map," which introduces specific topological advantages for environmental simulation.

6.1 Topological Advantages of Hexagonal Grids

In spatial simulations, particularly those involving isotropic propagation phenomena such as fluid dynamics or wildland fire spread, regular hexagonal grids provide distinct mathematical advantages over standard Cartesian (square) grids.⁶³ In a square grid, the distance from the centroid of a cell to its four orthogonally adjacent neighbors is exactly 1 , but the distance to its four diagonally adjacent neighbors is $\sqrt{2}$. This mathematical disparity creates

anisotropic artifacts in distance and speed calculations, causing simulated fires that should burn in a perfect circle under zero-wind conditions to unnaturally shape into diamonds or squares.⁶⁵

A regular hexagonal tessellation entirely eliminates this artifact. It ensures that the distance between the centroid of a central cell and the centroids of all six of its contiguous neighbors is perfectly identical.⁶⁴ This equidistant topology allows the fire propagation algorithm to spread uniformly in all directions without requiring complex mathematical weighting factors to compensate for diagonal displacement vectors.

6.2 Python Implementation and Cube Coordinate Systems

Generating and managing hexagonal grids efficiently on the Raspberry Pi 4 utilizes specialized Python libraries such as H3 (an open-source geospatial indexing system developed by Uber) or Geohexgrid.⁶⁸ To index and traverse the grid efficiently without relying on complex, staggered Cartesian offsets, the system employs a three-axis "cube coordinate" system (x, y, z) wherein the mathematical constraint $x + y + z = 0$ is perpetually maintained across the entire plane.⁷⁰ Each discrete hexagon in the grid serves as an isolated data container. As the rover navigates, the Python backend populates the grid cells corresponding to its current GPS localization. Each cell stores a multidimensional array containing the semantic data transmitted via LoRa:

1. **Elevation (Z_{elev}):** Derived from the MPU6050 gradient integration to determine slope.⁴⁶
2. **Fuel State (F_{state}):** A continuous variable mapping from 0.0 to 1.0 representing the presence and density of dry grass, derived from the OpenCV NGRDI pipeline.⁷²
3. **Wind Vector ($\vec{W}$):** Localized wind speed and the 8-point azimuth from the SS41F anemometer.⁷³
4. **Moisture Index (M_{idx}):** Derived from the DHT11 relative humidity telemetry.³²

7. Stochastic Cellular Automata for Wildfire Simulation

With the hexagonal grid populated by the rover's empirical data, the system transitions to its ultimate predictive function: simulating macroscopic fire propagation. The chosen computational framework is Cellular Automata (CA), an algorithmic modeling approach ideally suited for grid-based discrete-time dynamical systems.⁷⁴

7.1 State Transitions in the CA Model

Unlike continuous vector-based models utilizing Huygens' principle (such as standard FARSITE models) which calculate the continuous geometric expansion of a fire perimeter, CA models operate by iteratively updating the state of individual cells based entirely on the states of their immediate neighbors at discrete time steps ($t + \Delta t$).³⁵

In this specific architecture, each hexagonal cell i exists simultaneously in one of several discrete states:

- S_0 : Unburned (Combustible fuel present, $F_{\text{state}} > 0$).
- S_1 : Actively Burning.

- p_2 : Burned out / Ashes (non-combustible).
- p_3 : Non-combustible base (e.g., bare soil, rock, or areas identified by the camera as entirely devoid of dry vegetation).

The core mechanic of the simulation dictates that a burning cell (p_1) possesses a specific mathematical probability of igniting adjacent unburned cells (p_0).²²

7.2 The Propagation Probability Function

The transition rule governing fire spread is not purely deterministic; rather, it is a stochastic probability function heavily influenced by the localized environmental variables collected by the rover.⁷⁶ The specific probability p_{burn} that a burning cell will successfully ignite a neighboring cell in a given computational time step is calculated using a multiplicative weighting model³⁷:

$$p_{\text{burn}} = p_0 \cdot (1 + p_{\text{veg}}) \cdot p_w \cdot p_s$$

Each parameter within this equation is dynamically supplied by the rover's sensor suite:

- **p_0 (Base Probability)**: A constant baseline ignition probability under theoretical flat terrain, zero wind, and standard fuel conditions.
- **p_{veg} (Vegetation Factor)**: Directly derived from the OpenCV dry grass detection (F_{state}). A high density of cured grass increases this value, while sparse or healthy green vegetation (positive NGRDI) reduces it to near zero.⁶⁶ This factor is also inversely scaled by the moisture index (M_{idx}) from the DHT11 sensor; higher ambient humidity directly suppresses the value of p_{veg} .³²
- **p_w (Wind Factor)**: Wind drastically alters flame geometry, pushing convective heat columns toward unburned fuel. The wind factor utilizes the speed V (from the anemometer) and the angle θ between the prevailing wind direction vector and the spatial vector pointing from the burning cell to the target neighbor cell.⁶⁶ The relationship is modeled exponentially:

$$p_w = \exp \left(V \left(c_1 + c_2 (\cos(\theta) - 1) \right) \right)$$
 where c_1 and c_2 are empirically derived tuning coefficients. When the fire spreads in the exact direction the wind is blowing ($\theta = 0^\circ$), $\cos(0) = 1$, and the exponent is maximized, leading to rapid forward spread. When attempting to spread against the wind ($\theta = 180^\circ$), $\cos(180^\circ) = -1$, significantly dampening the propagation probability, effectively modeling flanking and backing fires.³⁷
- **p_s (Slope Factor)**: Derived from the MPU6050 elevation map (Z_{elev}). The slope angle α between the burning cell and the target cell modifies spread. Uphill spread exponentially increases p_s due to flame trenching and pre-heating of the canopy, while downhill spread decreases it.⁶⁶

7.3 Simulation Execution and Edge Performance

The CA simulation iteratively evaluates these probabilities across the entire hexagonal grid. The Raspberry Pi 4 is highly capable of running CA matrices in Python using heavily vectorized NumPy operations, allowing for computational simulations that project hours of fire spread in a

matter of seconds.⁷² By utilizing the real-time localized data gathered by the rover, the model serves as an "in-situ" predictive tool, generating immediate, high-resolution localized risk maps without relying on broad, low-resolution satellite data or requiring communication with external High-Performance Computing (HPC) cloud clusters.²²

8. Final Feasibility and Architectural Evaluation

The proposed 6WD autonomous rover system presents a highly sophisticated, multi-tiered approach to environmental data collection and hazard simulation. The synthesis of the hardware, software, and mathematical paradigms reveals several defining operational conclusions:

First, the integration of 6 DC motors, 6 servos, and 3 L298N motor drivers is mechanically robust but strictly requires the daisy-chaining of two PCA9685 controllers over the I2C bus to satisfy the channel requirements, alongside careful frequency separation for DC and servo actuation. Furthermore, the explicit inclusion of an ADC (such as an ADS1115) is an absolute necessity to digitize the analog telemetry from the MQ135 sensor, compensating for the Raspberry Pi's hardware limitations.

Second, the kinematic viability of the holonomic swerve configuration provides superior mobility for targeted sensor pointing. The DIY anemometer utilizing four SS41F Hall sensors offers an elegant, mathematically sound solution capable of providing precise 8-way directional vectors and RPM-based wind speeds crucial for the simulation logic. The Extended Kalman Filter fusion of the MPU6050 and Neo 6M GPS data successfully yields the micro-topographic gradient maps required to calculate the critical slope coefficient (ρ_s) in the CA equations. Third, while detecting dry grass utilizing HSV thresholds and NGRDI color space algorithms via OpenCV is computationally efficient for edge processing, the utilization of RF96 LoRa dictates a strict semantic telemetry pipeline. Raw video transmission is physically impossible over LoRa's limited bandwidth; the system must process all images onboard and transmit only the resulting numeric geospatial payloads (hex cell coordinates and corresponding environmental variables). Ultimately, by mapping this high-fidelity, edge-computed sensor data onto an isotropic hexagonal grid, the system provides the perfect matrix for Stochastic Cellular Automata. By mathematically linking the gathered variables to the transition probabilities, the rover successfully transitions from a passive observation platform into an autonomous, localized forecasting node capable of generating highly accurate, in-situ simulations of wildland fire behavior.

Sources des citations

1. Stair Climbing Rover by WildWilly | Download free STL model | Printables.com, consulté le avril 23, 2026, <https://www.printables.com/model/194299-stair-climbing-rover/collections>
2. WildWillyRobots - YouTube, consulté le avril 23, 2026, <https://www.youtube.com/@wildwillyrobots/videos>
3. Stair Climbing Rover by WildWilly | Download free STL model ..., consulté le avril 23, 2026, <https://www.printables.com/model/194299-stair-climbing-rover>

4. DTU RoboCup 2022 - Wild Willy - First run with onboard camera overlay - YouTube, consulté le avril 23, 2026, <https://www.youtube.com/watch?v=GkBIQkr3NZU>
5. Controlling DC motors and servos with a single pca9685 - Electronics Stack Exchange, consulté le avril 23, 2026, <https://electronics.stackexchange.com/questions/400647/controlling-dc-motors-and-servos-with-a-single-pca9685>
6. GitHub - custom-build-robots/Motor-Driver-L298N-and-PCA9685: With this set of python programs it is possible to use a PCA9685 servo controller board to control L298N H-Bridge., consulté le avril 23, 2026, <https://github.com/custom-build-robots/Motor-Driver-L298N-and-PCA9685>
7. Easy way!! Control Multiple Servo with Raspberry Pi - YouTube, consulté le avril 23, 2026, <https://www.youtube.com/watch?v=9jcEwn7GzNs>
8. Adafruit PCA9685 16-Channel Servo Driver, consulté le avril 23, 2026, <https://cdn-learn.adafruit.com/downloads/pdf/16-channel-pwm-servo-driver.pdf>
9. OSOYOO Pie Car V2.0 Lesson 1: Basic installation & programming GPIO/PCA9685 (C Version), consulté le avril 23, 2026, <https://osoyoo.com/2020/11/06/osoyoo-pie-car-v2-0-for-raspberry-pi-lesson-1-basic-framework-and-programming-gpio-pca9685-c-version/>
10. How to interface dc gear motor from L298n to pca9685 to arduino uno, consulté le avril 23, 2026, <https://forum.arduino.cc/t/how-to-interface-dc-gear-motor-from-l298n-to-pca9685-to-arduino-uno/958232>
11. PCA9685 - Multiple Servo Control Using Arduino, consulté le avril 23, 2026, <https://circuitdigest.com/microcontroller-projects/pca9685-multiple-servo-control-using-arduino>
12. How to connect dual PCA9685 servo controller to Raspberry pi?, consulté le avril 23, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=351863>
13. How to use PCA 9685 to drive variable speed DC Motor - Adafruit Forums, consulté le avril 23, 2026, <https://forums.adafruit.com/viewtopic.php?t=184030>
14. Air Quality Monitoring System for Hazardous Gas Detection and Fire Risk in a Chemical Laboratory, consulté le avril 23, 2026, <https://ioinformatic.org/index.php/JAIEA/article/download/2109/1424>
15. (PDF) Air Quality Monitoring Using MQ135 Gas Sensor and Arduino Uno - ResearchGate, consulté le avril 23, 2026, https://www.researchgate.net/publication/393126306_Air_Quality_Monitoring_Using_MQ135_Gas_Sensor_and_Arduino_Uno
16. Room-Climate measuring with MQ-135 and DHT11 - Hackster.io, consulté le avril 23, 2026, <https://www.hackster.io/az-delivery/room-climate-measuring-with-mq-135-and-dht11-39d4ac>
17. How to power 8 Servo Motors with PCA9685 and RPi4B? - Raspberry Pi Forums, consulté le avril 23, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=342296>
18. Rover build - Raspberry Pi Forums, consulté le avril 23, 2026,

- <https://forums.raspberrypi.com/viewtopic.php?t=200139>
19. Adafruit 16 Channel Servo Driver with Raspberry Pi, consulté le avril 23, 2026, <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-16-channel-servo-driver-with-raspberry-pi.pdf>
 20. Controlling DC Motor with Raspberry Pi and L298N motor driver - YouTube, consulté le avril 23, 2026, https://www.youtube.com/watch?v=OV0S_3KMj2A
 21. How to predict the spread and intensity of forest and range fires, consulté le avril 23, 2026, https://www.fs.usda.gov/rm/pubs_int/int_gtr143.pdf
 22. Fire Spread Modeling using Probabilistic Cellular Automata* - arXiv, consulté le avril 23, 2026, <https://arxiv.org/pdf/2403.08817>
 23. Wind speed - Build your own weather station - Raspberry Pi Foundation, consulté le avril 23, 2026, <https://projects.raspberrypi.org/en/projects/build-your-own-weather-station/5>
 24. DIY Anemometer With Hall-Sensor : 6 Steps (with Pictures) - Instructables, consulté le avril 23, 2026, <https://www.instructables.com/DIY-Anemometer-With-Hall-Sensor/>
 25. Hall-effect sensors are advantageous for use in harsh environments and space-constrained applications. They have fewer cost | - Automation | Honeywell, consulté le avril 23, 2026, <https://automation.honeywell.com/content/dam/honeywell-edam/sps/siot/en-us/campaigns/2025/rty-dual-output/documents/hon-ia-hss-hall-effect-theory-rotary-sensors-wp-en.pdf>
 26. Wind Speed and Direction Sensor: Anemometer DIY Arduino Build - Zbotic, consulté le avril 23, 2026, <https://zbotic.in/wind-speed-and-direction-sensor-anemometer-diy-arduino-build/>
 27. A DIY Wind Gauge (Anemometer) based on ESP32 - Smart Solutions for Home, consulté le avril 23, 2026, <https://smartsolutions4home.com/ss4h-wg-wind-gauge/>
 28. Homemade Anemometer, consulté le avril 23, 2026, <http://cs2.uwsuper.edu/sb/Electronics/Wind/>
 29. Near Real-Time Windspeed Tests. Part 2 - Fine Offset Anemometer ..., consulté le avril 23, 2026, <https://www.meteocercal.info/forum/thread-1381.html>
 30. AeroVane: IoT Environmental Station & DIY Wind Direction Sensor (Pico W) - Instructables, consulté le avril 23, 2026, <https://www.instructables.com/AeroVane-IoT-Environmental-Station-DIY-Wind-Direct/>
 31. How to Build Your Own Anemometer Using Reed Switches, Hall Effect Sensor and Some Scraps on Nodemcu. - Part 1 - Hardware - Instructables, consulté le avril 23, 2026, <https://www.instructables.com/How-to-Build-Your-Own-Anemometer-Using-Reed-Switch/>
 32. What is the Fire weather index? - Cordulus, consulté le avril 23, 2026, <https://www.cordulus.com/glossary/fire-weather-index>
 33. The Daily Fire Hazard Index: A Fire Danger Rating Method for Mediterranean

- Areas - MDPI, consulté le avril 23, 2026,
<https://www.mdpi.com/2072-4292/12/15/2356>
34. Understanding Fire Danger (U.S. National Park Service), consulté le avril 23, 2026,
<https://www.nps.gov/articles/understanding-fire-danger.htm>
 35. A Review of Two-Dimensional Cellular Automata Models for Wildfire Simulation: Methods, Capabilities, and Limitations - MDPI, consulté le avril 23, 2026,
<https://www.mdpi.com/2571-6255/9/3/108>
 36. Statistical dynamics of wildfire burned area from cellular-automata simulators - PMC, consulté le avril 23, 2026,
<https://pubmed.ncbi.nlm.nih.gov/articles/PMC12783638/>
 37. Modelling propagation of wildfires using cellular automata - Universidade de Lisboa, consulté le avril 23, 2026,
https://fenix.tecnico.ulisboa.pt/downloadFile/281870113705098/Extended_abstract_Barbara_Mota_vf.pdf
 38. (De)coding the MPU6050 and the Control Action - Duke MEMS, consulté le avril 23, 2026,
<https://sites.duke.edu/memscapstone/decoding-the-mpu6050-and-the-control-action/>
 39. Navigating using GPS Localization - Nav2 1.0.0 documentation, consulté le avril 23, 2026, https://docs.nav2.org/tutorials/docs/navigation2_with_gps.html
 40. Build a Digital Level with MPU-6050 and Arduino | DroneBot Workshop, consulté le avril 23, 2026, <https://dronebotworkshop.com/mpu-6050-level/>
 41. Measure Angles Easily with MPU6050 and ESP32: Part 1 - Library Walkthrough - YouTube, consulté le avril 23, 2026,
<https://www.youtube.com/watch?v=C93SoHYeuz0>
 42. Is it possible to determine MPU6050 mounting orientation programatically?, consulté le avril 23, 2026,
<https://robotics.stackexchange.com/questions/117128/is-it-possible-to-determine-mpu6050-mounting-orientation-programatically>
 43. Real-Time Localization and Mapping Utilizing Multi-Sensor Fusion and Visual-IMU-Wheel Odometry for Agricultural Robots in Unstructured, Dynamic and GPS-Denied Greenhouse Environments - MDPI, consulté le avril 23, 2026,
<https://www.mdpi.com/2073-4395/12/8/1740>
 44. Arduino and MPU6050 Accelerometer and Gyroscope Tutorial - How To Mechatronics, consulté le avril 23, 2026,
<https://howtomechatronics.com/tutorials/arduino/arduino-and-mpu6050-accelerator-and-gyroscope-tutorial/>
 45. Using the gravity vector provided by the IMU of the robots, the maps... - ResearchGate, consulté le avril 23, 2026,
https://www.researchgate.net/figure/Using-the-gravity-vector-provided-by-the-IMU-of-the-robots-the-maps-are-projected-onto_fig2_312294270
 46. Continuous On-Board Monocular-Vision-based Elevation Mapping Applied to Autonomous Landing of Micro Aerial Vehicles - Robotics and Perception Group, consulté le avril 23, 2026, https://rpg.ifi.uzh.ch/docs/ICRA15_Forster.pdf
 47. Semantic Mapping for Autonomous Navigation and Exploration - Carnegie Mellon

- University's Robotics Institute, consulté le avril 23, 2026,
https://www.ri.cmu.edu/app/uploads/2021/08/cmuthesis_template.pdf
48. SMART FIRE DETECTION SYSTEM USING OPENCV AND RASPBERRY PI - IJESAT, consulté le avril 23, 2026,
https://www.ijesat.com/ijesat/files/V17I1001_1750316904.pdf
49. Using a Standard RGB Camera and AI to Obtain Vegetation Data, consulté le avril 23, 2026,
<https://siebelschool.illinois.edu/news/using-a-standard-rgb-camera-and-ai-to-obtain-vegetation-data>
50. Camera-based obstacle/lawn detection (experimental) - Ardumower Forum, consulté le avril 23, 2026,
<https://forum.ardumower.de/threads/camera-based-obstacle-lawn-detection-experimental.25095/>
51. Implementing real-time wildfire detection using lightweight object-detection models and machine vision sensor on Raspberry Pi 5: - ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences, consulté le avril 23, 2026,
<https://isprs-annals.copernicus.org/articles/X-2-W2-2025/81/2025/isprs-annals-X-2-W2-2025-81-2025.pdf>
52. Yashaswini278/Grass-Lane-Detection - GitHub, consulté le avril 23, 2026,
<https://github.com/Yashaswini278/Grass-Lane-Detection>
53. Colour-based object tracking with Raspberry Pi, consulté le avril 23, 2026,
<https://www.raspberrypi.com/news/colour-based-object-tracking-with-raspberrypi/>
54. RGB vegetation indices applied to grass monitoring: a qualitative analysis - CABI Digital Library, consulté le avril 23, 2026,
<https://www.cabidigitallibrary.org/doi/pdf/10.5555/20203349950>
55. yard inside garden detection (cv2) - python - Stack Overflow, consulté le avril 23, 2026,
<https://stackoverflow.com/questions/62886284/yard-inside-garden-detection-cv2>
56. Calculation of the vegetation indices (VIs) in order to estimate the crop health of the under-study field. - GitHub, consulté le avril 23, 2026,
<https://github.com/CoFly-Project/Vegetation-Indices>
57. Calculate NDVI & Extract Spectra Using Masks in Python | NSF NEON, consulté le avril 23, 2026,
<https://www.neonscience.org/resources/learning-hub/tutorials/calc-refl-ndvi-py>
58. GitHub - hallard/LoRasPI: Raspberry PI Lora Gateway/Node for HopeRF RFM95 RFM96 RFM98 Modules, consulté le avril 23, 2026,
<https://github.com/hallard/LoRasPI>
59. Use Lora Shield and RPi to Build a LoRaWAN Gateway - Instructables, consulté le avril 23, 2026,
<https://www.instructables.com/Use-Lora-Shield-and-RPi-to-Build-a-LoRaWAN-Gateway/>
60. Using LoRa for communication between ESP32 and Raspberry Pi | Jay Gould,

- consulté le avril 23, 2026,
<https://jaygould.co.uk/2023-05-03-high-altitude-balloon-tracking-lora-point-to-point/>
61. Controlling Pi over Long Distances - Raspberry Pi Forums, consulté le avril 23, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=218288>
 62. Edge computing driven forest fire spread simulation: An energy-aware study, consulté le avril 23, 2026,
<https://portalrecerca.uab.cat/en/publications/edge-computing-driven-forest-fire-spread-simulation-an-energy-awa/>
 63. (PDF) An efficient method to simulate wildfire propagation using irregular grids, consulté le avril 23, 2026,
https://www.researchgate.net/publication/395020564_An_efficient_method_to_simulate_wildfire_propagation_using_irregular_grids
 64. HexFire: A Flexible and Accessible Wildfire Simulator - PMC, consulté le avril 23, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9534040/>
 65. Using cellular automata to simulate wildfire propagation and to assist in fire prevention and fighting - Opiware, consulté le avril 23, 2026,
[https://opiware.it/wp-content/uploads/WMTS_Bibliografia/nhess-2018-227%20\(1\).pdf](https://opiware.it/wp-content/uploads/WMTS_Bibliografia/nhess-2018-227%20(1).pdf)
 66. Using cellular automata to simulate wildfire propagation and to assist in fire prevention and fighting - NHES, consulté le avril 23, 2026,
<https://nhess.copernicus.org/preprints/nhess-2018-227/nhess-2018-227.pdf>
 67. HexFire: A Flexible and Accessible Wildfire Simulator - ResearchGate, consulté le avril 23, 2026,
https://www.researchgate.net/publication/362631989_HexFire_A_Flexible_and_Accessible_Wildfire_Simulator
 68. arachev/geohegrid: A Python library for making geographic flat-top hexagon grids like QGIS's `create grid` function - GitHub, consulté le avril 23, 2026,
<https://github.com/arachev/geohegrid>
 69. Constructing Hexagon Maps with H3 and Plotly: A Comprehensive Tutorial, consulté le avril 23, 2026,
<https://towardsdatascience.com/constructing-hexagon-maps-with-h3-and-plotly-a-comprehensive-tutorial-8f37a91573bb/>
 70. HexMap Library Documentation: HexMapLibrary, consulté le avril 23, 2026,
<https://aurelwu.github.io/>
 71. Implementation of Hex Grids - Red Blob Games, consulté le avril 23, 2026,
<https://www.redblobgames.com/grids/hexagons/implementation.html>
 72. Simulating Wildfires in a Forest | by Aleksei Rozanov | The Pythoneers - Medium, consulté le avril 23, 2026,
<https://medium.com/pythoneers/simulating-wildfires-in-a-forest-e0550c28035a>
 73. Using cellular automata to simulate wildfire propagation and to assist in fire management, consulté le avril 23, 2026,
<https://nhess.copernicus.org/articles/19/169/>
 74. Forest Fire Spread Simulation and Fire Extinguishing Visualization Research - MDPI, consulté le avril 23, 2026, <https://www.mdpi.com/1999-4907/14/7/1371>

75. An Efficient Method to Simulate Wildfire Propagation Using Irregular Grids - NHESS, consulté le avril 23, 2026,
<https://nhess.copernicus.org/preprints/nhess-2024-27/nhess-2024-27.pdf>
76. Clustered-map probabilistic cellular automata for fire propagation in the Brazilian Cerrado with heterogeneous vegetation and wind interference - AIMS Press, consulté le avril 23, 2026,
<https://www.aimspress.com/article/doi/10.3934/urs.2024004?viewType=HTML>